

SNOWLINK

IMU + GNSS Logging and Fusion Implementation Plan

Physical setup, application architecture, algorithm choices, implementation plan, and phase-gate test checklists

Prepared for the Snowlink rover project - 2026-05-26

Document purpose

This document is intended as a field and development reference. Use it before sensor mounting, before logging, before running GNSS/IMU fusion, and before moving to the next implementation phase. It is written around your stated hardware: a u-blox ZED-F9P/F9P GNSS module and a WHEELTEC N100 IMU.

Contents

1. One-page implementation summary
2. System scope and design principles
3. Hardware and physical sensor setup
4. Coordinate frames, mounting offsets, and map origin
5. Time, timestamps, and logging strategy
6. ZED-F9P/F9P GNSS setup
7. WHEELTEC N100 IMU setup
8. Application architecture and module structure
9. Data formats and configuration files
10. Real-time fusion algorithm: ESKF
11. Map display integration
12. Implementation phases and deliverables
13. Test procedures and phase-gate checklists
14. Troubleshooting guide
15. Future expansion: LiDAR and offline smoothing
16. Reference notes from the two PDFs
17. Glossary

1. One-page implementation summary

The first Snowlink localization milestone should be a stable, replayable GNSS + IMU system that logs data, fuses it into a rover pose, and displays the correct rover location and heading on the map. Do not add LiDAR localization until this foundation is stable.

Part	Decision for Snowlink	Reason
Main real-time localization algorithm	Error-State Kalman Filter (ESKF)	Best fit for IMU prediction + GNSS correction while estimating small error states and IMU biases.
GNSS role	Absolute position correction	ZED-F9P gives global position and RTK quality information.
IMU role	High-rate motion prediction	N100 accelerometer and gyroscope bridge the gaps between GNSS updates.
Map coordinates	Local ENU: east, north, up	Filter and map operations are easier and numerically cleaner than using latitude/longitude directly.
Timestamp baseline	Computer monotonic timestamp for v1; improve later if needed	Simple and practical. Validate jitter with timestamp diagnostics.
GNSS trust model	Adaptive covariance by RTK quality	RTK fixed, RTK float, single GNSS, and invalid fixes must not be trusted equally.
LiDAR now?	No localization fusion yet	Use LiDAR later after pose foundation is correct.
Offline smoothing	Optional RTS later, not for live control	Useful for post-processing test data, not necessary for first live map marker.

Target v1 data flow:

ZED-F9P GNSS -> GNSS parser -> ENU conversion -> GNSS update -> ESKF pose -> Map marker
WHEELTEC N100 -> IMU parser -> IMU prediction -> ESKF pose -> Map marker

Debug overlays:

Raw GNSS dot + fused rover icon + RTK status + covariance/accuracy circle

2. System scope and design principles

- Make the first version boring and reliable. It should log, replay, and show the rover on the map before doing advanced autonomy.
- Separate raw sensor reading, logging, coordinate conversion, fusion, and GUI display into separate modules.
- Do not use latitude/longitude inside the filter. Convert GNSS to local ENU first.
- Keep raw GNSS and fused pose visible at the same time during testing. This helps detect when the filter is worse than GNSS alone.
- Treat RTK status as a measurement-quality signal, not just a UI label.
- Use the N100 built-in attitude as a diagnostic/reference, but base the fusion on raw gyro and accelerometer data.
- Make every test replayable without sensors connected. Log first, optimize later.

Phase boundary

Phase 1 is GNSS/IMU only. Phase 2 can add LiDAR mapping using the fused pose. Phase 3 can add LiDAR localization or graph-based SLAM. Do not collapse these phases, because debugging sensor fusion becomes very difficult when everything is added at once.

3. Hardware and physical sensor setup

3.1 Sensor inventory

Device	Project role	Main outputs to use	Notes
u-blox ZED-F9P/F9P GNSS	Absolute positioning and RTK quality	Latitude, longitude, height, fix type, RTK status, horizontal/vertical accuracy, satellites	Prefer UBX messages when possible because they are structured and include quality data.
WHEELTEC N100 IMU	High-rate inertial prediction	Acceleration, angular velocity, optional roll/pitch/yaw or quaternion	Use raw accel/gyro for fusion. Log built-in attitude for comparison.
Raspberry Pi / host computer	Data acquisition, filter, map display	System time, serial ports, GUI process	Timestamp jitter depends on OS load and serial parsing design.
GNSS antenna	Receives satellite signals	N/A	Mount high, rigid, and away from obstructions or strong RF noise.
Power and cables	Stable operation	Voltage and serial connectivity	Loose cables cause intermittent errors that look like software bugs.

3.2 Physical mounting checklist

- ☐ GNSS antenna is mounted on a rigid part of the rover, not on a flexible plastic cover.
- ☐ GNSS antenna has the clearest possible sky view.
- ☐ GNSS antenna is not directly beside high-current motor wires, DC/DC converters, radio transmitters, or metal shielding.
- ☐ IMU is mounted rigidly to the rover body, with no foam or soft tape that allows vibration movement.
- ☐ IMU orientation axes are documented with a photo and drawing.
- ☐ IMU is as close as practical to the rover body reference point or center of rotation.
- ☐ All cables have strain relief so that connector movement does not appear as sensor noise.
- ☐ Power supply is stable when motors start and when the rover turns.
- ☐ USB/serial device names are made stable using udev rules or a serial-number based mapping.
- ☐ The physical offsets between `base_link`, `imu_link`, and `gnss_link` are measured and entered into `config/sensor_offsets.yaml`.

3.3 Mounting offsets to measure

Measure these offsets with a ruler or caliper and record them in meters. Use the same sign convention as the rover body frame. Recommended body frame: x forward, y left, z up. If your application uses a different convention, document it clearly and convert at the sensor interface boundary.

Offset	Meaning	Required for v1?	How accurate should it be?
<code>base_link -> imu_link</code>	IMU position relative to rover body origin	Yes	Aim for +/- 5 mm to 10 mm if possible.
<code>base_link -> gnss_link</code>	GNSS antenna phase-center approximation relative to body origin	Yes	Aim for +/- 10 mm to 20 mm.
<code>imu_link</code> rotation	How IMU axes are rotated relative to rover body axes	Yes	Must be correct. A sign error can break the filter.
gnss antenna height	Vertical offset of GNSS antenna above body origin	Yes	Important for 3D position and future LiDAR mapping.
lidar_link offsets	For future LiDAR mapping	Later	Measure now if LiDAR is already mounted.

Example sensor_offsets.yaml:

```
frames:
  base_link:
    convention: "x forward, y left, z up"
  imu_link:
    parent: base_link
    translation_m: [0.00, 0.00, 0.05]
    rotation_rpy_deg: [0.0, 0.0, 0.0]
  gnss_link:
    parent: base_link
    translation_m: [0.10, 0.00, 0.35]
    rotation_rpy_deg: [0.0, 0.0, 0.0]
```

4. Coordinate frames, mounting offsets, and map origin

Frame	Meaning	Used by
map / enu	Local fixed East-North-Up frame. Origin is selected near test area.	Filter, map display, logs, overlays.
base_link	Rover body frame. Define x forward, y left, z up.	Control, visualization, transforms.
imu_link	N100 IMU measurement frame.	IMU parser and ESKF prediction.
gnss_link	GNSS antenna frame / antenna reference point.	GNSS lever-arm compensation.
screen/map pixels	GUI coordinate system for drawing map marker.	Map rendering only. Do not use inside filter.

The application should use latitude/longitude only at the GNSS interface and map tile interface. The filter should consume ENU position in meters.

GNSS processing pipeline:

lat/lon/height -> ECEF -> local ENU -> GNSSMeasurementENU -> ESKF update -> FusedPose -> map display

4.1 Map origin rules

- [] For early development, set ENU origin from the first good RTK_FIXED point.
- [] For repeatable field tests, use a saved fixed origin in config/map_origin.yaml.
- [] Do not change the map origin while a log or test run is active.
- [] Store the origin in every log directory so runs can be replayed later.
- [] If the map marker is offset from the visual map, verify the origin, coordinate order, and map projection before blaming the filter.

5. Time, timestamps, and logging strategy

Every sensor sample must have a timestamp. For the first implementation, use time.monotonic() when the message is received by the application. Also log the GNSS time if available. Later, you can improve the system with GNSS time, PPS, or hardware timestamping if test results show the need.

Timestamp type	Use in v1?	Purpose	Caution
monotonic receive time	Yes	Primary timestamp for ordering and dt in v1.	Includes serial and OS delay, so monitor jitter.
system wall time	Yes, for logs	Human-readable log correlation.	Can jump if system time changes. Do not use for integration dt.

GNSS time of week / UTC	Log if available	Useful for debugging and future time alignment.	Requires correct parsing and leap-second handling.
hardware timestamp / PPS	Later	Best for tighter multi-sensor synchronization.	More wiring and implementation complexity.

5.1 Logging principles

- ☐ Log raw sensor data before filtering.
- ☐ Log converted ENU GNSS data after coordinate conversion.
- ☐ Log fused pose after every filter output update.
- ☐ Log sensor status and error messages, not only numeric sensor samples.
- ☐ Each test run gets a new folder named with date/time and test name.
- ☐ Each test run stores the config files used during that run.
- ☐ Replay should use the log timestamps, not the current wall clock.

Recommended log directory structure:

```
logs/
2026-05-26_static_test_001/
  config_snapshot/
    map_origin.yaml
    sensor_offsets.yaml
    filter_params.yaml
  imu_raw.csv
  gnss_raw.csv
  gnss_enu.csv
  fused_pose.csv
  system_status.csv
  notes.md
```

6. ZED-F9P/F9P GNSS setup

The ZED-F9P should be treated as the absolute position sensor. For fusion, the important output is not just position, but position plus quality. The filter must know whether the GNSS measurement is RTK fixed, RTK float, single GNSS, invalid, or stale.

Data item	Required?	Use
latitude, longitude, height	Yes	Convert to ENU position.
fix type	Yes	Reject invalid fixes and classify GNSS quality.
carrier solution / RTK status	Yes	Different covariance for RTK fixed, RTK float, and no RTK.
horizontal accuracy	Yes if available	Set or validate measurement covariance.
vertical accuracy	Yes if available	Set z measurement covariance.
number of satellites	Recommended	Display and diagnostics.
HDOP/PDOP	Recommended	Diagnostics and quality checks.
GNSS time	Recommended	Future timestamp validation.

6.1 GNSS receiver configuration checklist

- ☐ ZED-F9P baud rate is set and matches the application config.
- ☐ Only needed messages are enabled to avoid serial overload.
- ☐ RTK correction source is configured and verified before fusion testing.
- ☐ Antenna cable is secure and not damaged.
- ☐ GNSS status reports RTK_FIXED in open-sky conditions before starting filter tests.

- ☐ GNSS update rate is known and stable. Start with 1 Hz or 5 Hz.
- ☐ The application displays last GNSS age in seconds.
- ☐ The application rejects stale GNSS measurements.

Example gnss config:

```
gnss:
  device: "/dev/serial/by-id/zed_f9p"
  baudrate: 115200
  protocol: "ubx"
  preferred_messages: ["NAV-PVT", "NAV-HPPOSLLH", "NAV-COV"]
  stale_timeout_s: 2.0
  set_origin_from_first_rtk_fixed: true
```

7. WHEELTEC N100 IMU setup

The N100 should be treated as the high-rate motion sensor. Use raw accelerometer and gyroscope values for the ESKF. If the N100 provides roll/pitch/yaw or quaternion, log it and show it in diagnostics, but do not let it silently replace the filter state during the first fusion implementation.

Data item	Required?	Use
accel_x, accel_y, accel_z	Yes	Specific force input for IMU prediction.
gyro_x, gyro_y, gyro_z	Yes	Angular velocity input for orientation propagation.
roll/pitch/yaw or quaternion	Optional	Diagnostics and initialization reference only in v1.
temperature	Optional	Useful for drift analysis.
IMU status flags	If available	Reject bad packets or unhealthy sensor state.

7.1 IMU configuration checklist

- ☐ N100 baud rate is set and matches the application config.
- ☐ Acceleration units are converted to m/s².
- ☐ Gyro units are converted to rad/s.
- ☐ IMU axes are mapped to the rover body convention.
- ☐ Stationary acceleration magnitude is close to 9.81 m/s².
- ☐ Stationary gyro mean is close to zero after bias estimation.
- ☐ IMU sample rate is stable and displayed in diagnostics.
- ☐ Parser detects checksum or packet framing errors if the protocol provides them.

Example imu config:

```
imu:
  device: "/dev/serial/by-id/wheeltec_n100"
  baudrate: 115200
  frame: "imu_link"
  accel_units: "mps2"
  gyro_units: "radps"
  expected_rate_hz: 100
  stationary_calibration_seconds: 120
```

8. Application architecture and module structure

```
snowlink/  
  config/  
    map_origin.yaml  
    sensor_offsets.yaml  
    filter_params.yaml  
  sensors/  
    gnss_reader.py  
    imu_reader.py  
  logging/  
    csv_logger.py  
    replay_logger.py  
  localization/  
    coordinate_transform.py  
    timestamp_monitor.py  
    imu_calibration.py  
    eskf.py  
    gnss_update.py  
    pose_publisher.py  
  gui/  
    map_view.py  
    diagnostics_view.py  
  tests/  
    test_coordinate_transform.py  
    test_imu_parser.py  
    test_gnss_parser.py  
    test_eskf_replay.py
```

Module	Responsibility	Must not do
gnss_reader.py	Read and parse ZED-F9P data. Publish raw GNSS measurement object.	Do not run the filter.
imu_reader.py	Read and parse N100 data. Publish raw IMU measurement object.	Do not convert map coordinates.
coordinate_transform.py	Convert geodetic to ECEF to ENU.	Do not draw GUI elements.
timestamp_monitor.py	Measure rates, jitter, stale data, dropped packets.	Do not modify sensor data silently.
eskf.py	Predict from IMU and update from GNSS.	Do not read serial ports directly.
pose_publisher.py	Convert filter state into FusedPose and log it.	Do not change filter state.
map_view.py	Draw raw GNSS dot and fused rover marker.	Do not implement sensor parsing.

9. Data formats and configuration files

9.1 Python data objects

```
from dataclasses import dataclass  
  
@dataclass  
class ImuMeasurement:  
    t: float  
    ax: float  
    ay: float  
    az: float  
    gx: float
```

```

gy: float
gz: float
roll: float | None = None
pitch: float | None = None
yaw: float | None = None

```

```

@dataclass
class GnssMeasurement:
    t: float
    lat: float
    lon: float
    height: float
    fix_type: str
    rtk_status: str
    h_acc: float | None
    v_acc: float | None
    satellites: int | None

```

```

@dataclass
class GnssEnuMeasurement:
    t: float
    east: float
    north: float
    up: float
    fix_type: str
    rtk_status: str
    h_acc: float | None
    v_acc: float | None

```

```

@dataclass
class FusedPose:
    t: float
    east: float
    north: float
    up: float
    velocity_east: float
    velocity_north: float
    velocity_up: float
    roll: float
    pitch: float
    yaw: float
    gnss_status: str
    filter_status: str
    position_covariance: list[float]

```

9.2 CSV log schemas

File	Columns
imu_raw.csv	timestamp_monotonic,timestamp_unix,ax,ay,az,gx,gy,gz,roll,pitch,yaw,temp,status
gnss_raw.csv	timestamp_monotonic,timestamp_unix,lat,lon,height,fix_type,rtk_status,h_acc,v_acc,satellites,hdop
gnss_enu.csv	timestamp_monotonic,east,north,up,fix_type,rtk_status,h_acc,v_acc
fused_pose.csv	timestamp_monotonic,east,north,up,ve,vn,vu,roll,pitch,yaw,gnss_status,filter_status,cov_x,cov_y,cov_z

system_status.csv	timestamp_monotonic,component,level,message
-------------------	---

10. Real-time fusion algorithm: ESKF

Use an Error-State Kalman Filter for live GNSS + IMU fusion. The nominal state represents the current best estimate of position, velocity, orientation, and IMU biases. The error state represents small corrections. GNSS updates correct the error state, the correction is injected into the nominal state, and the error is reset.

10.1 Nominal state

State part	Symbol	Description
Position	$p = [e, n, u]$	Rover position in local ENU meters.
Velocity	$v = [ve, vn, vu]$	Rover velocity in ENU meters per second.
Orientation	q	Quaternion from body to ENU. Display as roll/pitch/yaw.
Accelerometer bias	ba	Slowly changing accelerometer bias.
Gyroscope bias	bg	Slowly changing gyro bias.

10.2 IMU prediction step

For every IMU sample:

1. $dt = imu.t - previous_imu.t$
2. $gyro_corrected = gyro - gyro_bias$
3. $accel_corrected = accel - accel_bias$
4. $q = integrate_orientation(q, gyro_corrected, dt)$
5. $accel_enu = rotate(q, accel_corrected) + gravity_correction$
6. $v = v + accel_enu * dt$
7. $p = p + v * dt + 0.5 * accel_enu * dt^2$
8. $P = propagate_covariance(P, F, Q, dt)$

10.3 GNSS correction step

When a GNSS ENU measurement arrives:

1. Reject if stale, invalid, or impossible.
2. $R = gnss_covariance_from_status(rtk_status, h_acc, v_acc)$
3. $innovation = z_gnss - predicted_gnss_position$
4. Reject if innovation is too large for the current covariance.
5. Compute Kalman gain.
6. Update error state.
7. Inject error into nominal state.
8. Reset error state.
9. Log fused_pose and filter diagnostics.

10.4 Adaptive GNSS covariance

GNSS status	Recommended behavior	Example covariance
RTK_FIXED	Strong correction	$R = diag(0.02^2, 0.02^2, 0.05^2)$
RTK_FLOAT	Medium correction	$R = diag(0.20^2, 0.20^2, 0.50^2)$
3D / single fix	Weak correction	$R = diag(2.0^2, 2.0^2, 5.0^2)$
No fix / invalid	Skip update	None
Stale GNSS	Skip update	None

Important tuning rule

The numbers above are starting values only. Tune them using static and driving tests. If the fused position jumps toward bad GNSS, increase R. If the filter ignores good RTK, decrease R.

10.5 Initialization rules

- [] Wait for a valid GNSS position before initializing position.
- [] Prefer RTK_FIXED for the first origin and first position initialization.
- [] Initialize roll/pitch from stationary IMU gravity if practical.
- [] Initialize yaw from GNSS course only after the rover has moved enough, or use the N100 yaw as a temporary display/debug value.
- [] Do not claim accurate yaw while the rover is stationary unless you have a trusted heading source.
- [] Start with zero velocity if rover is stationary during initialization.
- [] Run stationary bias calibration before the first driving test.

11. Map display integration

During development, the map should display both raw GNSS and fused pose. This is essential for debugging. The fused marker should become the main rover icon only after the GNSS-only map has been verified.

Map element	Meaning	Source
Blue dot	Raw GNSS ENU position	gnss_enu.csv / live GNSS
Rover icon	Fused ESKF pose	FusedPose
Heading arrow	Yaw from fused pose	ESKF orientation
Accuracy circle	Approximate horizontal covariance	$\sqrt{\text{cov_e} + \text{cov_n}}$
Status label	RTK fixed/float/single/no fix	GNSS parser
Age label	Time since last GNSS and IMU sample	timestamp_monitor.py

Map marker logic during testing:

```
if filter_status == "READY":
    draw_rover_icon(fused_pose.east, fused_pose.north, fused_pose.yaw)
    draw_accuracy_circle(fused_pose.position_covariance)
else:
    draw_gnss_debug_dot(gnss.east, gnss.north)

always_draw_raw_gnss_dot_when_debug_enabled = True
```

12. Implementation phases and deliverables

Phase	Build	Deliverable	Gate to pass
1	GNSS reader + logging	gnss_raw.csv with valid position and status	Open-sky RTK_FIXED is visible and stable.
2	ENU conversion + raw map marker	Blue GNSS dot on map	Marker follows correct physical movement.
3	IMU reader + logging	imu_raw.csv and diagnostics screen	Stationary accel ≈ 9.81 m/s ² , gyro stable.
4	Timestamp monitor	Rate, jitter, age, stale warnings	No unexpected gaps during static test.
5	ESKF prediction + GNSS update	fused_pose.csv and rover icon	Fused pose smoother than raw GNSS and no large jumps.
6	Replay mode	Run filter from logs	Replay produces same result as live run.
7	Field validation	Static, straight, stop-start, dropout tests	Pass phase-gate checklist.

13. Test procedures and phase-gate checklists

13.1 Pre-test field checklist

- ☐ Battery is charged and power wiring is secure.
- ☐ GNSS antenna is connected before powering the GNSS module.
- ☐ IMU and GNSS serial devices appear with stable names.
- ☐ RTK correction source is connected if required for this test.
- ☐ Application config matches the current physical mounting.
- ☐ `map_origin.yaml` is correct for the test site, or `origin-from-first-RTK-fixed` is enabled.
- ☐ Log folder is created with a meaningful test name.
- ☐ Raw GNSS and raw IMU logging are enabled.
- ☐ Fused pose logging is enabled if testing fusion.
- ☐ System clock and storage space are checked.
- ☐ Weather and sky visibility notes are written in `notes.md`.
- ☐ E-stop or safe manual control is available if rover will move.

13.2 Phase 1 checklist: GNSS logging

- ☐ GNSS parser receives data continuously.
- ☐ Latitude/longitude/height are reasonable for the test site.
- ☐ Fix type is displayed.
- ☐ RTK fixed/float/single status is displayed.
- ☐ Horizontal and vertical accuracy are displayed if available.
- ☐ Satellite count is displayed.
- ☐ `gnss_raw.csv` contains timestamp, position, status, and accuracy fields.
- ☐ Application warns if GNSS is stale.
- ☐ GNSS log can be replayed without GNSS connected.

13.3 Phase 2 checklist: ENU conversion and map marker

- ☐ ENU origin is stored in the log folder.
- ☐ GNSS ENU values are in meters and start near zero if origin is first point.
- ☐ East increases when moving east.
- ☐ North increases when moving north.
- ☐ Up value is reasonable and not wildly drifting in open sky.
- ☐ Raw GNSS marker appears at expected location on the map.
- ☐ Map marker follows a simple walking or driving path.
- ☐ Accuracy circle changes based on GNSS quality or configured accuracy.
- ☐ Map display does not crash if GNSS is temporarily invalid.

13.4 Phase 3 checklist: IMU logging and diagnostics

- ☐ IMU parser receives continuous packets.
- ☐ IMU frequency matches expected rate within tolerance.
- ☐ Acceleration units are m/s^2 .
- ☐ Gyro units are rad/s .
- ☐ Stationary acceleration magnitude is close to 9.81 m/s^2 .

- ☐ Stationary gyro values are near zero after bias estimation.
- ☐ Roll/pitch direction matches physical tilting.
- ☐ Yaw/heading is clearly marked as diagnostic unless trusted.
- ☐ imu_raw.csv contains all raw values and timestamps.
- ☐ IMU log can be replayed without IMU connected.

13.5 Phase 4 checklist: first ESKF fusion

- ☐ Filter initializes only after valid GNSS position is available.
- ☐ Filter uses IMU dt from monotonic timestamps.
- ☐ Filter rejects negative dt and unreasonably large dt.
- ☐ Covariance grows when GNSS is unavailable.
- ☐ Covariance shrinks when good RTK GNSS updates arrive.
- ☐ RTK_FIXED uses stronger correction than RTK_FLOAT.
- ☐ Invalid or stale GNSS is skipped.
- ☐ Raw GNSS dot and fused rover icon are both visible in debug mode.
- ☐ Fused pose does not jump violently when GNSS status changes.
- ☐ Filter can be reset from the UI or command line.
- ☐ fused_pose.csv contains position, velocity, attitude, GNSS status, and covariance.

13.6 Static test procedure

Step	Action	Expected result
1	Place rover outdoors with clear sky.	GNSS reaches RTK_FIXED or best available status.
2	Start logging for 10 minutes without moving rover.	Position scatter is visible but bounded.
3	Run IMU bias estimator.	Gyro bias becomes stable; accel magnitude near 9.81 m/s ² .
4	Start ESKF.	Fused position should be smoother than raw GNSS.
5	Review logs.	No timestamp gaps, no serial parser errors.

13.7 Straight-line test procedure

Step	Action	Expected result
1	Move in a straight line 20-50 m at low speed.	Raw GNSS and fused pose follow same line.
2	Stop at the end for 30 s.	Velocity approaches zero.
3	Return along same path.	Fused path returns close to start.
4	Review heading arrow.	Heading points approximately in direction of travel when moving.

13.8 Stop-start and GNSS dropout test

- ☐ Drive or push rover slowly, stop for 10 s, repeat 5 times.
- ☐ Verify velocity estimate approaches zero at stops.
- ☐ In replay, remove GNSS updates for 2 s, 5 s, and 10 s.
- ☐ Verify position covariance grows during simulated dropout.
- ☐ Verify filter resumes smoothly when GNSS returns.
- ☐ If filter jumps after GNSS returns, inspect GNSS covariance and innovation gating.

13.9 Phase gate: before moving to LiDAR mapping

- ☐ GNSS-only map position works reliably.
- ☐ IMU logging and diagnostics are stable.
- ☐ ESKF fused pose is stable in static, straight-line, and stop-start tests.
- ☐ Replay mode works.
- ☐ Physical sensor offsets are measured and stored.
- ☐ Filter parameters are saved with each log.
- ☐ There is a known procedure to reset/reinitialize the filter.
- ☐ Raw GNSS and fused pose can be compared visually on the map.
- ☐ No unresolved sign convention issues remain.
- ☐ No unexplained timestamp gaps remain.

14. Troubleshooting guide

Symptom	Likely cause	What to check
Map marker appears in wrong place	Wrong map origin, lat/lon order, projection issue	Check origin, coordinate conversion, and raw GNSS values.
East/north movement reversed	Axis convention mismatch	Check ENU conversion and body frame convention.
Filter position jumps on GNSS update	GNSS covariance too small or bad GNSS accepted	Increase R, check RTK status, add innovation gate.
Fused pose drifts quickly without GNSS	IMU bias/noise, wrong gravity compensation, wrong axes	Run static calibration and axis tests.
Yaw is wrong while stationary	GNSS course cannot define heading without motion	Do not trust course heading at zero speed.
IMU data looks noisy with motors on	Vibration or power noise	Improve mounting, isolate power, inspect cable routing.
GNSS loses RTK near rover electronics	RF interference or antenna placement	Move antenna, improve ground plane, separate from power electronics.
Replay does not match live run	Using current time instead of logged timestamps	Fix replay runner to use log timestamps.
Serial device changes name	USB enumeration order changed	Use /dev/serial/by-id or udev rules.
Filter crashes after pause	Large dt not handled	Clamp/reject large dt and reinitialize if needed.

15. Future expansion: LiDAR and offline smoothing

After GNSS/IMU fusion passes the phase gate, use the fused pose to transform LiDAR scans into the map frame. Initially, LiDAR should be a mapping and perception sensor, not a localization source. Later, LiDAR scan matching can be added as an additional correction source.

Future feature	When to add	Notes
LiDAR terrain/snow grid	After ESKF pose is stable	Use pose interpolation at LiDAR timestamp.
LiDAR scan matching correction	After LiDAR map is stable	Only correct filter when scan matching quality is high.
RTS smoothing	Offline analysis	Do not depend on it for live rover control.
Graph-based SLAM	Longer-term	Useful when you need loop closure and tighter LiDAR/IMU/GNSS optimization.
Hardware timestamping / PPS	If timing tests show software jitter problem	Adds complexity but can improve determinism.

16. Reference notes from the two PDFs

This plan is based on your Snowlink hardware choice and on practical lessons from the two uploaded PDFs. The PDF references below are summarized as engineering guidance rather than copied as implementation code.

Source	Relevant takeaway for Snowlink
Yin et al., 2023 - Sensor Fusion of GNSS and IMU Data for Robust Localization via Smoothed Error State Kalman Filter	Use an ESKF structure for GNSS + IMU fusion: IMU prediction, GNSS update, error injection, reset, and covariance propagation. RTS smoothing is useful for offline improvement but should not be required for live map display.
Storli, 2023 - Timing and time synchronization within LiDAR- and IMU-based SLAM	Treat timing, coordinate frames, mounting offsets, and lever arms as first-class design items. Software timestamps can be acceptable in some setups but must be measured, not assumed.
Storli, 2023 - GNSS/IMU/LiDAR mounting and GTSAM modifications	GNSS antenna and IMU rarely occupy the same physical point. Measure offsets and compensate for lever arms to avoid systematic position errors.

17. Glossary

Term	Meaning
ENU	East, North, Up local coordinate frame in meters.
ESKF	Error-State Kalman Filter. Estimates small corrections around a nominal nonlinear state.
RTK fixed	High-confidence RTK GNSS solution. Usually the best live GNSS correction status.
RTK float	Carrier-phase RTK solution without fixed integer ambiguities. Usually less accurate than RTK fixed.
Covariance	Mathematical uncertainty representation. Larger covariance means less confidence.
Innovation	Difference between predicted measurement and actual measurement.
Lever arm	Physical offset between sensor origins, such as IMU and GNSS antenna.
Monotonic time	Clock that only moves forward; good for measuring dt in software.
Quaternion	Orientation representation used internally to avoid Euler angle singularities.
Replay	Running the application from logged data instead of live sensors.